

Pooling choices for probing count representations

A practical memo for the dolphin-marker Dynamic NIAH experiments

Summary

The core recommendation is to treat within-example pooling and across-example occurrence averaging as complementary diagnostics, not substitutes. For predicting the total count of a prompt, the training unit is the example, so the most natural probe features are one vector per example: pooled marker-occurrence states, last-marker states, final-query states, or answer-prefix states. Across-example averages at fixed occurrence index are useful for understanding whether the model has a running-count geometry - for example, whether the first, second, and third occurrences form distinct prototypes - but they do not directly produce a per-example count predictor.

In a causal language model, an early marker occurrence cannot attend to future occurrences. Therefore, the first occurrence should not be expected to encode the final count, except through correlations. It is more plausible that occurrence states encode local identity and possibly a running index, while later positions such as the last marker, post-marker tokens, query token, or answer-prefix token are candidates for consolidated total-count information.

1. What the counting paper's entity-mean probe tests

The paper *The Right Answer, the Wrong Direction: Why Transformers Fail at Counting and How to Fix It* studies whether a model fails because it lacks count information internally or because the count direction is poorly read out by digit-token logits. The paper reports that linear probes can recover counts with very high accuracy, while the count directions are nearly orthogonal to digit output rows. It also explicitly compares an entity-mean position with the last-token position in its logit-lens analysis. [1]

For a fixed example i and layer l , let S_i be the set of token positions occupied by the target entity/marker occurrences. The entity-mean residual activation is one pooled vector per example:

$$\begin{aligned} \bar{h}[i, l] &= (1 / |S_i|) * \sum_{t \in S_i} h[i, l, t] \\ \hat{y}_i &= w[l]^T \bar{h}[i, l] + b[l] \end{aligned}$$

This answers: if we aggregate the representations of the counted items in this prompt, is the total count y_i linearly decodable? It is a diagnostic representation. It is not literally the same hidden state that the model's output head uses during ordinary generation.

2. Two different averaging axes

Averaging operation	Object produced	Question answered	Best use
Within each example over marker positions: $\text{mean}_t h[i, l, t]$	One vector per prompt	Does the set/multiset of marker states in this prompt contain the total count?	Ridge/regression prediction of y_i ; diagnostic of local marker-state information.
Across examples at fixed occurrence index: $\text{mean}_i h[i, l, t_{\{i\}}]$	One prototype per occurrence index j	Does the j -th marker occurrence have a distinct representation?	Geometry and running-count diagnostics; not directly a per-example predictor.
Final/query/answer-prefix state $h[i, l, T]$	One vector per prompt at the readout-relevant location	Has count information been routed to the position that produces the answer?	Closest to generation; useful for readout-bottleneck tests.

Your suggestion to average across examples at each key position is valuable, but it estimates occurrence-index prototypes rather than a predictor. For example, $\mu_1, \mu_2, \mu_3, \dots$ can reveal whether the model distinguishes first, second, third occurrences. It should be paired with, not replace, per-example pooling for total-count prediction.

3. Why within-example pooling is still natural for total count

The label y_i is an example-level label: the total number of marker occurrences in the entire prompt. For this kind of target, the clean supervised design is one row per prompt. A pooled marker representation gives exactly that. This is related to a standard idea in set representation: when a target depends on an unordered collection of objects, a permutation-invariant pooling operation is a natural first diagnostic. Deep Sets characterizes invariant set functions using pooled element representations, and Set Transformer generalizes this idea with attention-based pooling for set-structured inputs. [2,3]

However, mean pooling is not always the best pooling operation for literal repeated markers. If every occurrence of [dolphin] has a similar representation, mean pooling can remove count magnitude. Sum pooling may preserve the number of occurrences more directly. Mean pooling can still work if occurrence states themselves encode running index information, such as first occurrence, second occurrence, and so on.

```
Case A: identical occurrence states
h[i,j] ~ a + noise
mean_j h[i,j] ~ a          # count may be erased
sum_j h[i,j] ~ k_i * a    # count magnitude preserved

Case B: running-count occurrence states
h[i,j] ~ a + j*v + noise
mean_j h[i,j] ~ a + ((k_i+1)/2)*v # mean predicts k_i
```

4. What across-example occurrence averaging is good for

Across-example averaging at fixed occurrence index is ideal for asking whether the model has a running-count representation. Define $t_{i,j}$ as the j -th marker occurrence in example i . Then:

```
mu[j,l] = mean_{i: count_i >= j} h[i,l,t_{i,j}]
```

Plotting or comparing $\mu[1,l]$, $\mu[2,l]$, $\mu[3,l]$, ... can show whether occurrence index is represented in a low-dimensional direction. But because $\mu[j,l]$ averages across examples, it loses the individual prompt identity. It is therefore a population-level diagnostic, not a direct predictor for an unseen prompt's total count.

A stronger version is to fit occurrence-level probes. For each occurrence j in each example, train one probe to predict the running index j and another to predict the final total y_i . This separates local ordinal information from consolidated total-count information.

5. Recommended probe grid for the dolphin-marker task

Feature family	Definition	Prediction target	Interpretation
marker_all_mean	Mean over all tokens in all [dolphin] spans	total count y_i	Paper-like entity-mean diagnostic; denoises local context.
marker_last_mean	Mean over final token of each marker span	total count y_i	Occurrence-summary version; avoids bracket-token dilution.
marker_last_sum	Sum over final token of each marker span	total count y_i	Tests additive accumulator/magnitude hypothesis.
occurrence_j_state	h at the j -th marker occurrence	running index j and total y_i	Distinguishes ordinal/running count from final total.
last_marker_state	h at the last marker occurrence	total count y_i	Tests whether total is available by the final occurrence.
post_marker_mean	Mean over token immediately after each marker	total count y_i	Tests whether count is written after reading each marker.
final_query / answer_prefix	h at query or answer-prefix location	total count y_i	Tests routing to answer position and readout bottleneck.

Layer and metric grid

- Layers: start with layers 8, 12, 16, 20, 24, 28 for Qwen3-8B; then densify around the best layer.
- Counts: use a full count ladder, such as 0, 1, 2, 3, 4, 5, 6, 8, 10, 12, not only count 3.
- Metrics: report R^2 , rounded-count accuracy, mean absolute error, and a candidate-count logprob ranking for generated answers.
- Controls: include position-matched non-marker spans and label-shuffled probes; probe selectivity matters because high-capacity probes can learn artifacts. Hewitt and Liang's control-task framework is a useful warning here. [4]

6. Interpretation matrix

Observed result	Interpretation	Next action
High marker-pooling R^2 , low final-token R^2	The model detects/counts occurrences locally, but does not consolidate the total at the answer position.	Study routing: attention from query/answer positions to marker spans; activation patch pooled/last-marker states into final-query states.
High final-token R^2 , wrong generation	Classic geometric readout bottleneck: the correct count is present but not aligned with digit/JSON output tokens.	Run digit-logit/lm-head alignment, candidate-count logprob, and output-head/logit-lens analyses.
High $h[i,j] \rightarrow j$ but low $h[i,j] \rightarrow y_i$	Occurrences encode running ordinal position but not the final total at each local marker state.	Probe last-marker, post-marker, and answer-prefix states; test whether later layers consolidate.
Low marker-pooling R^2 and low final-token R^2	The task may fail at marker detection or stable enumeration, not only output readout.	Verify spans/tokenization, run marker-vs-nonmarker detection probes, shorten context, and inspect error cases.
Sum pooling succeeds but mean pooling fails	Each occurrence contributes a roughly additive vector; mean erased count magnitude.	Use sum pooling for diagnostic probes and test additive steering/ablation.
Mean pooling succeeds but sum pooling is unstable	Occurrence states may encode running index or context-normalized count information.	Analyze occurrence-index prototypes $\mu[j,l]$ and directions $\mu[j+1,l] - \mu[j,l]$.

7. Practical implementation sketch

The following pseudo-code assumes you have verified marker spans in final model-token coordinates. It intentionally produces one feature vector per example for the total-count probe.

```
# H_l: [T, d] hidden states for one example at layer l
# marker_spans: list of (start, end) token spans for [dolphin]
marker_all = concat([H_l[s:e] for (s,e) in marker_spans])
marker_last = stack([H_l[e-1] for (s,e) in marker_spans])
post_marker = stack([H_l[e] for (s,e) in marker_spans if e < T])

features = {
    'marker_all_mean': marker_all.mean(axis=0),
    'marker_all_sum': marker_all.sum(axis=0),
    'marker_last_mean': marker_last.mean(axis=0),
    'marker_last_sum': marker_last.sum(axis=0),
    'last_marker': marker_last[-1],
    'post_marker_mean': post_marker.mean(axis=0),
    'final_query': H_l[final_query_pos],
    'answer_prefix': H_l[answer_prefix_pos],
}
# Fit ridge separately for each feature family and layer.
# Train on varied counts; evaluate on held-out prompts/haystacks.
```

8. Suggested analysis sequence

First, verify that the model sees the marker reliably: train a marker-vs-nonmarker binary probe and inspect tokenization of [dolphin]. Second, fit total-count probes using marker_all_mean, marker_last_mean, marker_last_sum, last_marker, final_query, and answer_prefix. Third, build occurrence-index prototypes $\mu[j,l]$ and test whether $h[i,j]$ predicts running

index j . Fourth, compare the best probe results with candidate-count logprob rankings. This tells you whether the bottleneck is occurrence detection, local counting, routing to answer position, or output-readout alignment.

Given your observed error pattern for count 3 - mostly predicting 2 - the dominant failure looks like missing one occurrence rather than counting instruction/query mentions. That makes occurrence-level and last-marker analyses particularly important.

9. Recommended reporting table for each run

Column	Meaning
feature_family	marker_all_mean, marker_last_sum, final_query, etc.
layer	Transformer layer index.
train_counts / eval_counts	Count values used for training and held-out evaluation.
R2	Regression R^2 on held-out examples.
rounded_acc	Accuracy after rounding predicted count to nearest integer.
MAE	Mean absolute error of predicted count.
candidate_logprob_rank	Rank of correct JSON/count candidate under teacher forcing.
baseline_generation_acc	Ordinary greedy-generation accuracy for the same examples.
span_validation_rate	Fraction of examples with verified exact marker spans.

References

- [1] Gabriel Garcia. The Right Answer, the Wrong Direction: Why Transformers Fail at Counting and How to Fix It. arXiv:2605.03258, 2026. The paper reports highly accurate linear count probes and analyzes entity-mean vs last-token positions and output-head/digit-row misalignment.
- [2] Manzil Zaheer, Satwik Kottur, Siamak Ravanbakhsh, Barnabas Poczos, Ruslan Salakhutdinov, Alexander Smola. Deep Sets. NeurIPS 2017 / arXiv:1703.06114. Characterizes permutation-invariant set functions using pooled element representations.
- [3] Juho Lee, Yoonho Lee, Jungtaek Kim, Adam R. Kosiorek, Seungjin Choi, Yee Whye Teh. Set Transformer: A Framework for Attention-based Permutation-Invariant Neural Networks. ICML 2019 / arXiv:1810.00825. Generalizes set pooling with attention-based set encoders/decoders.
- [4] John Hewitt and Percy Liang. Designing and Interpreting Probes with Control Tasks. EMNLP 2019. Emphasizes probe selectivity and the need for controls when interpreting whether information is encoded in representations.

Checklist for the next experiment

- Generate a count ladder, not only count 3, with verified marker spans for every example.
- Compare mean vs sum pooling over marker occurrences, and compare both to final-query/answer-prefix states.
- Fit occurrence-index probes $h[i,j] \rightarrow j$ to test running-count geometry.
- Report both probe metrics and generation metrics; a good probe is diagnostic, not automatically causal.
- Use matched non-marker spans and label-shuffled controls before interpreting high R^2 as representational evidence.